

Experiment # 02

Python Lists, Tuples, Dictionaries and Function

Objective

1. To understand basics of Python Lists, Tuples and Dictionaries.
2. Implementing Python codes for lists, tuples and dictionaries.
3. Implementing Python Functions.

Software

- Python Idle 3.7 or 3.8.5
- Pycharm

Theory

1. Lists

Lists are the most versatile of Python's compound data types. A list contains items separated by commas and enclosed within square brackets ([]). To some extent, lists are similar to arrays in C. One difference between them is that all the items belonging to a list can be of different data type.

The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end(length-1). The plus (+) sign is the list concatenation operator, and the asterisk (*) is the repetition operator. For

Example # 01:

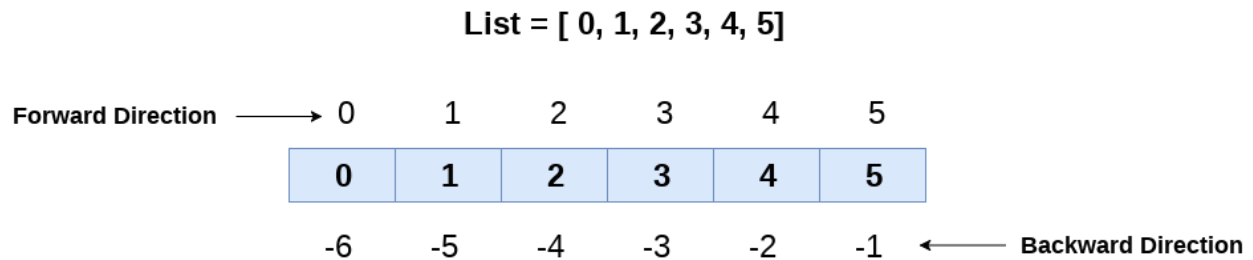
```
emp = ["Usama", 102, "PAKISTAN"]
Dep1 = ["CS",10];
Dep2 = ["IT",11];
HOD_CS = [10,"Mr. Junaid"]
HOD_IT = [11, "Mr. Saad"]
print("Printing Employee Data...");
print("Name : %s, ID: %d, Country: %s"%(emp[0],emp[1],emp[2]))
print("printing departments...");
print("Department 1:\nName: %s, ID: %d\nDepartment 2:\nName: %s, ID: %s"
      %(Dep1[0],Dep2[1],Dep2[0],Dep2[1]));
print("HOD Details ....");
```

```
print("CS HOD Name: %s, Id: %d" %(HOD_CS[1],HOD_CS[0]));
```

```
print("IT HOD Name: %s, Id: %d" %(HOD_IT[1],HOD_IT[0]));
```

```
print(type(emp), type(Dep1), type(Dep2), type(HOD_CS), type(HOD_IT));
```

Unlike other languages, python provides us the flexibility to use the negative indexing also. The negative indices are counted from the right. The last element (right most) of the list has the index-1, its adjacent left element is present at the index-2 and so on until the left most element is encountered.



Python List Operations

The concatenation (+) and repetition (*) operator work in the same way as they were working with the strings.

Let's see how the list responds to various operators.

1. Consider a List L1 = [1, 2, 3, 4], **and** L2 = [5, 6, 7, 8]

Operator	Description	Example
Repetition	The repetition operator enables the list elements to be repeated multiple times.	L1*2 = [1, 2, 3, 4, 1, 2, 3, 4]
Concatenation	It concatenates the list mentioned on either side of the operator.	L1+L2 = [1, 2, 3, 4, 5, 6, 7, 8]
Membership	It returns true if a particular item exists in a particular list otherwise false.	print(2 in L1) prints True.
Iteration	The for loop is used to iterate over the list elements.	for i in L1: print(i)
Length	It is used to get the length of the list	len(L1) = 4

Slicing	list [start: end: step size] start is inclusive and end is exclusive defaults values are list [0 : end : 1]	<pre>print (L1 [1:3]) print (L1 [0:4:2]) print (L1 [:]) # print (L1) print (L1 [0:]) print (L1 [:3])</pre>
----------------	--	--

Python List Built-in-methods

- **Index()**

The **index()** methods searches for an element in a list. For instance:

```
my_list = ['a','b','c','b', 'a']
```

It returns the index of the first and only occurrence of 'b'.

- **Append()**

Python provides append() function by using which we can add an element to the list.

However, the append() method can only add the value to the end of the list.

```
my_list.append(5)
```

- **Insert()**

You can insert element wherever you want in a list:

```
my_list.insert(2, 'a')
```

The **insert()** methods insert an object before the index provided.

- **Remove()**

Similarly, you can remove the first occurrence of an element as follows:

```
my_list.remove('a')
```

- **Pop()**

Or remove the last element of a list by using:

```
my_list.pop()
```

which also returns the value that has been removed.

- **Count()**

You can count the number of element of a kind:

```
my_list.count('b')
```

Example # 02:

```
L = ['a' , 'b' , 'w' , 'x' , 'z']  
L.insert(0,'x')  
print(L)  
L.remove('a')  
print(L)  
L.pop()  
print(L)  
count = L.count('x')  
print(count)  
L.reverse()  
print (L)
```

Python List Built-in functions

Python provides the following built-in functions which can be used with the lists.

SN	Function	Description
1	len(list)	It is used to calculate the length of the list.
2	max(list)	It returns the maximum element of the list.
3	min(list)	It returns the minimum element of the list.
5	list(seq)	It converts any sequence to the list.

2. Tuples

A tuple is another sequence data type that is similar to the list. A tuple consists of a number of values separated by commas. Unlike lists, however, tuples are enclosed within parentheses.

The main differences between lists and tuples are:

Lists are enclosed in brackets ([]) and their elements and size can be changed, while tuples are enclosed in parentheses (()) and cannot be updated. Tuples can be thought of as read-only lists.

Example:

```
T1 = (101, "Ayush", 22)
```

```
T2 = ("Apple", "Banana", "Orange")
```

```
T3 = ( 'abcd' , 786 , 67.554 , “Pakistan” , ‘H’)
```

```
print( len (T3))
```

```
print( type (T3[0]), type (T3[1]), type (T3[2]), type (T3[3]), type (T3[4]))
```

SN	List	Tuple
1	The literal syntax of list is shown by the [].	The literal syntax of the tuple is shown by the ().
2	The List is mutable.	The tuple is immutable.
3	The List has the variable length.	The tuple has the fixed length.
4	The list provides more functionality than tuple.	The tuple provides less functionality than the list.
5	The list Is used in the scenario in which we need to store the simple collections with no constraints where the value of the items can be changed.	The tuple is used in the cases where we need to store the read-only collections i.e., the value of the items cannot be changed. It can be used as the key inside the dictionary.

Example # 03:

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
```

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
```

```
tuple[2] = 1000 # Invalid syntax with tuple
```

```
list[2] = 1000 # Valid syntax with list
```

Where to use tuple

Using tuple instead of list is used in the following scenario.

- Using tuple instead of list gives us a clear idea that tuple data is constant and must not be changed.

- ii. Tuple can be used as the key inside dictionary due to its immutable nature.

Python Tuple Built-in functions

Python provides the following built-in functions which can be used with the tuples.

SN	Function	Description
1	len(tuple)	It is used to calculate the length of the tuple.
2	max(tuple)	It returns the maximum element of the tuple.
3	min(tuple)	It returns the minimum element of the tuple.
4	tuple(seq)	It converts any sequence to the tuple.
5	tuple.count(1)	It counts elements in tuple.

3. Dictionaries:

Dictionary is used to implement the key-value pair in python. The dictionary is the data type in python which can simulate the real-life data arrangement where some specific value exists for some particular key.

In other words, we can say that a dictionary is the collection of key-value pairs where the value can be any python object whereas the keys are the immutable python object, i.e., Numbers, string or tuple.

Dictionaries are enclosed by curly braces ({ }) and values can be assigned and accessed using square braces ([]).

Creating the dictionary

The dictionary can be created by using multiple key-value pairs enclosed with the small brackets () and separated by the colon (:). The collections of the key-value pairs are enclosed within the curly braces {}.

The syntax to define the dictionary is given below.

```
Dict = {"Name": "Bilal", "Age": 30}
```

Built-in Dictionary functions

The built-in python dictionary methods along with the description are given below.

SN	Function	Description
1	if(dict1==dict2)	It compares the items of both the dictionary and returns true if the first dictionary values are greater than the second dictionary, otherwise it returns false.
2	len(dict)	It is used to calculate the length of the dictionary.
3	str(dict)	It converts the dictionary into the printable string representation.
4	type(variable)	It is used to print the type of the passed variable.

4. Functions

Functions are the most important aspect of an application. A function can be defined as the organized block of reusable code which can be called whenever required.

Python allows us to divide a large program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the python program.

In other words, we can say that the collection of functions creates a program. The function is also known as procedure or subroutine in other programming languages.

Python provide us various **built-in** functions like range() or print(). Although, the user can create its functions which can be called **user-defined functions**.

Creating a function

In python, we can use **def** keyword to define the function. The syntax to define a function in python is given below.

```
def my_function():  
    function-suite  
    return <expression>
```

The function block is started with the colon (:) and all the same level block statements remain at the same indentation. A function can accept any number of parameters that must be the same in the definition and function calling.

Function calling

In python, a function must be defined before the function calling otherwise the python interpreter gives an error. Once the function is defined, we can call it from another function or the python prompt. To call the function, use the function name followed by the parentheses.

A simple function that prints the message "Hello Word" is given below.

```
def hello_world():  
    print("hello world")  
  
hello_world()
```

Lab Task

1. Write a Python program to check if the given number is a Disarium Number. A number is said to be the Disarium number when the sum of its digit raised to the power of their respective positions becomes equal to the number itself.

For example, 175 is a Disarium number as follows:

$$1^1 + 7^2 + 5^3 = 1 + 49 + 125 = 175$$

2. Write a Python program to count the elements in a list until an element is a tuple.
Hint: Use `isinstance(object, type)`
3. Count the number of strings where the string length is 2 or more and the first and last character are same from a given list of strings.

Sample List: ['abc', 'xyz', 'aba', '1221', 'xyzzyx', 'aa', '122']

Expected Result: 4

4. Find the sum and product of numbers in a tuple.

Sample List: (1 , 2 , 4 , 2 , 6)

Sum is: 15

Product is: 96